

Práctica 3: Estructuras de datos para la Inteligencia Artificial

Materia: Estructura de datos

Hecho por: Ashley Dayanne Rodríguez Acosta

Ejercicio 4: Historial de Entrenamiento de un Modelo

Problema: Un sistema de aprendizaje automático registra las métricas de precisión de un modelo...

Objetivo: Permitir al usuario agregar la precisión de cada época...

Conceptos a reforzar: Listas dinámicas, inserción de elementos, bucles y búsqueda del valor máximo en una lista.

Algoritmo:

1. Crear una lista vacía llamada precisiones.
2. Pedir al usuario ingresar la precisión de cada época.
3. Mientras la precisión sea mayor o igual a 0, agregarla a la lista.
4. Si la lista no está vacía, obtener el último valor como precisión final y calcular la máxima.
5. Mostrar los resultados; en caso contrario, mostrar mensaje de lista vacía.

Código:

```
# Historial de entrenamiento de un modelo

precisiones = []

precision = int(input("Ingresa la precisión de cada época (ingresa un número negativo para salir): "))

while precision >= 0:
    precisiones.append(precision)
    precision = int(input("Ingresa la precisión de cada época (ingresa un número negativo para salir): "))

if len(precisiones) > 0:
    precision_final = precisiones[-1]
    precision_maxima = max(precisiones)
    print("Precisión final: ", precision_final, "\n Precisión máxima: ", precision_maxima)
else:
    print("No se ingresó ningún valor.")
```

Conclusión: En este ejercicio se reforzó la utilidad de las listas dinámicas...

Ejercicio 5: Sistema de Recomendación de Películas con Matrices

Problema: Un sistema de recomendación utiliza una matriz...

Objetivo: Imprimir matriz, calcular promedio de una película específica y mostrar calificaciones de un usuario.

Conceptos a reforzar: Matrices, acceso a filas y columnas completas, cálculo de promedios.

Algoritmo:

1. Definir la matriz con las calificaciones.
2. Recorrer filas y columnas para imprimir la matriz.
3. Calcular la suma y promedio de una película seleccionada.
4. Mostrar las calificaciones de un usuario específico.
5. Recorrer nuevamente la matriz mostrando usuario, película y calificación.

Código:

```
# Ejercicio 5: Sistema de recomendación de películas con matrices
matriz = [
    [5, 5, 4, 5, 5, 1, 5, 5, 3, 5, 3, 4, 4],
    [2, 5, 4, 5, 5, 3, 5, 4, 5, 5, 3, 5, 3],
    [2, 5, 5, 5, 4, 3, 5, 4, 4, 5, 2, 4, 3],
    [4, 5, 4, 5, 5, 4, 4, 5, 5, 5, 3, 4, 2],
    [5, 4, 4, 5, 5, 4, 5, 5, 3, 5, 1, 5, 1],
    [3, 5, 3, 5, 3, 5, 2, 1, 3, 5, 3, 1, 1],
    [2, 4, 5, 3, 5, 2, 5, 3, 3, 4, 1, 3, 2],
    [4, 5, 5, 2, 5, 1, 3, 5, 5, 4, 3, 1, 3],
    [3, 2, 5, 2, 5, 2, 3, 2, 5, 4, 5, 4, 2],
    [3, 5, 2, 5, 3, 1, 1, 5, 1, 5, 3, 1, 5],
    [5, 1, 5, 5, 5, 4, 4, 1, 4, 5, 4, 1, 5],
    [5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5],
    [5, 4, 3, 5, 5, 1, 4, 2, 3, 5, 2, 1, 5]
]

# Imprimir la matriz de calificaciones
for i in range(len(matriz)):
    for j in range(len(matriz[i])):
        print(matriz[i][j], end="\t")
    print()

# Calificación de una película
suma = 0
pelicula = 10

for i in range(len(matriz)):
    suma += matriz[i][pelicula]
print("la suma de calificaciones película es: ", suma)
promedio = suma / len(matriz)
print("El promedio de calificaciones es: ", promedio)

calificacionesUsuario = matriz[0]
print("Calificaciones del usuario 0: ", calificacionesUsuario)

# Lógica para mostrar calificaciones
for i in range(len(matriz)):
    for j in range(len(matriz[i])):
        print("Usuario ", i, "Película ", j, ": ", matriz[i][j])
```

Conclusión: Se aprendió a manipular matrices en contextos de recomendación...

Ejercicio 6: Análisis de Ventas Diarias con Arreglos

Problema: Una tienda en línea registra las ventas de cada día de la semana...

Objetivo: Ingresar ventas, mostrar total, día con más y menos ventas.

Conceptos a reforzar: Arreglos unidimensionales, recorrido secuencial, búsqueda de máximos y mínimos.

Algoritmo:

1. Definir arreglo con los días y arreglo con ventas.
2. Imprimir las ventas de cada día.
3. Calcular el total de ventas con suma.
4. Identificar el día con más y menos ventas usando máximo y mínimo.

Código:

```
# Análisis de ventas diarias con arreglos
dias = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"]
ventas = [2580, 2770, 7595, 3264, 4500, 9372, 7205]

# Total ventas por día
print(f"Total vendido por día:\n {dias[0]}: {ventas[0]}\n {dias[1]}: {ventas[1]}\n {dias[2]}: {ventas[2]}")

# Total de ventas de la semana
total_ventas = sum(ventas)
print(f"Total de ventas de la semana: ", total_ventas)

mas_ventas = max(ventas)
dia_max = dias[ventas.index(mas_ventas)]
print(f"El día con más ventas fue el día {dia_max}")

menos_ventas = min(ventas)
dia_min = dias[ventas.index(menos_ventas)]
print(f"El día con menos ventas fue el día {dia_min}")
```

Conclusión: Se comprendió la importancia de los arreglos unidimensionales para almacenar y analizar información simple, así como el uso de operaciones de suma y búsqueda de extremos en datos.

Ejercicio 7: Tablero de Juego con Matriz

Problema: Representar un tablero 5x5 con obstáculos en una matriz.

Objetivo: Permitir al usuario llenar el tablero, mostrarlo y contar obstáculos totales y por fila.

Conceptos a reforzar: Matrices, recorrido por filas y columnas, conteo de elementos.

Algoritmo:

1. Crear matriz que representa el tablero.
2. Imprimir la matriz completa.
3. Recorrer cada fila y celda contando obstáculos totales.
4. Solicitar una fila específica y contar los obstáculos en ella.

Código:

```
# Tablero de juego con matriz
tablero = [
    [1, 0, 0, 0, 1],
    [0, 1, 0, 0, 1],
    [0, 0, 0, 1, 0],
    [1, 0, 1, 0, 0],
    [0, 0, 1, 1, 1]
]

# Mostrar el tablero en forma de matriz
print(f"Tablero del juego: {tablero}")

# Contar cuántos obstáculos hay en total
obstaculos = 0
for fila in tablero:
    for i in fila:
        if i == 1:
            obstaculos += 1
print(f"En el tablero existen {obstaculos} obstáculos")

# Contar cuántos obstáculos hay en una fila en específico
fila_elegida = int(input("Ingrese el número de la fila que desea contar (0-4)"))
obstaculos_fila = 0
for i in tablero[fila_elegida]:
    if i == 1:
        obstaculos_fila += 1
print(f"En la fila {fila_elegida} existen {obstaculos_fila} obstáculos")
```

Conclusión: Este ejercicio permitió aplicar el manejo de matrices como representaciones gráficas de entornos, reforzando cómo recorrer datos y contar elementos en estructuras bidimensionales.

Ejercicio 8: Calificaciones de Estudiantes con Matriz y Vectores

Problema: Cada estudiante presenta 3 exámenes y se almacenan en una matriz.

Objetivo: Registrar calificaciones, calcular promedios por estudiante y por examen, determinar máxima calificación.

Conceptos a reforzar: Matrices, uso conjunto con arreglos unidimensionales, cálculo de promedios y máximos.

Algoritmo:

1. Solicitar cantidad de estudiantes.
2. Para cada estudiante, ingresar calificaciones y almacenarlas en la matriz.
3. Imprimir la matriz de calificaciones.
4. Calcular promedio de cada estudiante y mostrarlo.
5. Calcular promedio de cada examen recorriendo columnas.
6. Identificar la calificación más alta y el estudiante que la obtuvo.

Código:

```
# Calificaciones de Estudiantes con Matriz y Vectores

calificaciones = []
# Ingresar cantidad de estudiantes
n = int(input("Ingrese el número de estudiantes: "))

# Crear la matriz de calificaciones
for i in range(n):
    fila = []
    print(f"Estudiante {i+1}:")
    for j in range(3):
        calificacion = float(input(f"Ingrese la calificación del examen {j+1}: "))
        fila.append(calificacion)
    calificaciones.append(fila)

# Mostrar matriz de calificaciones
print("\nMatriz de calificaciones:")
print(calificaciones)

# Calcular y mostrar el promedio de cada estudiante
promedios = []
for fila in calificaciones:
    promedio = sum(fila) / len(fila)
    promedios.append(promedio)

print("\nPromedio de cada estudiante:")
for i in range(len(promedios)):
    print(f"Estudiante {i+1}: {promedios[i]:.2f}")

# Calcular promedio de cada examen (columna)
promedio_exámenes = []
for j in range(3):
    suma = 0
    for i in range(n):
        suma += calificaciones[i][j]
    promedio = suma / n
    promedio_exámenes.append(promedio)

print("\nPromedio de cada examen:")
for j in range(3):
    print(f"Examen {j+1}: {promedio_exámenes[j]:.2f}")

# Determinar qué estudiante obtuvo la calificación más alta en el curso
max_nota = 0
estudiante_max = 0
for i in range(n):
    for j in range(3):
```

```
if calificaciones[i][j] > max_nota:
    max_nota = calificaciones[i][j]
    estudiante_max = i + 1
```

```
print(f"\nEl estudiante {estudiante_max} obtuvo la calificación: {max_nota}, siendo esta la más alta del
```

Conclusión: Se integró el uso de matrices y vectores para resolver un problema de análisis académico, reforzando la capacidad de recorrer filas y columnas, calcular promedios y determinar máximos de forma eficiente.